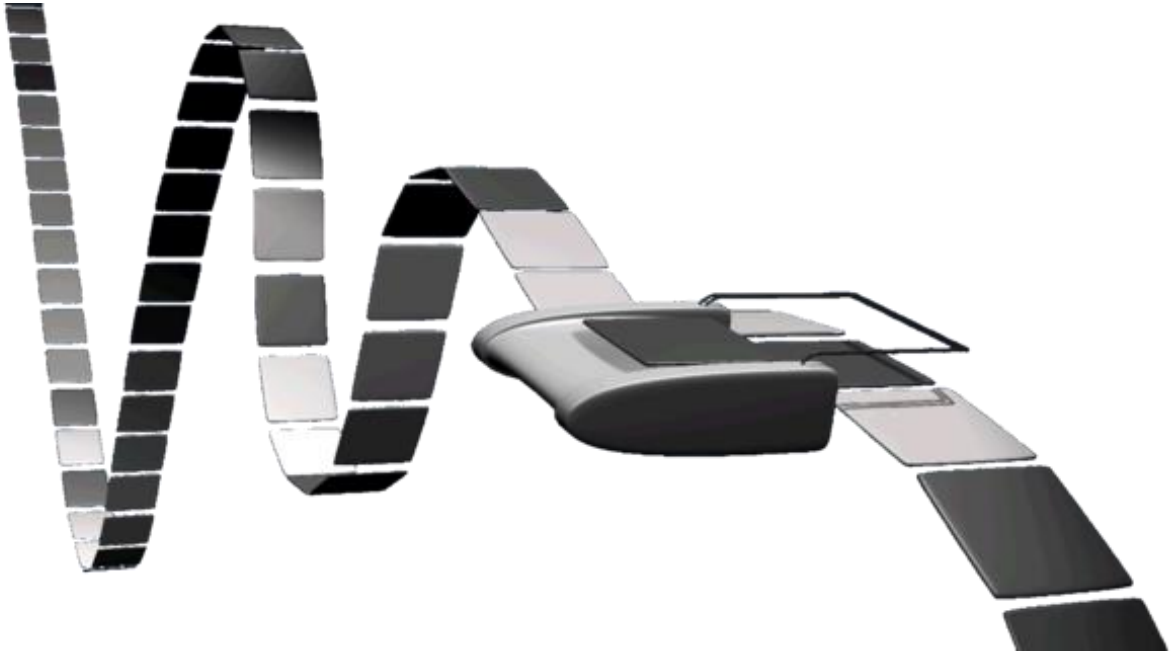
A VERY SHORT INTRODUCTION TO THE UNSOLVED PROBLEMS OF COMPUTER SCIENCE & ENGINEERING

Professor Ahmet KOLTUKSUZ, Ph.D.

A. COMPUTER SCIENCE

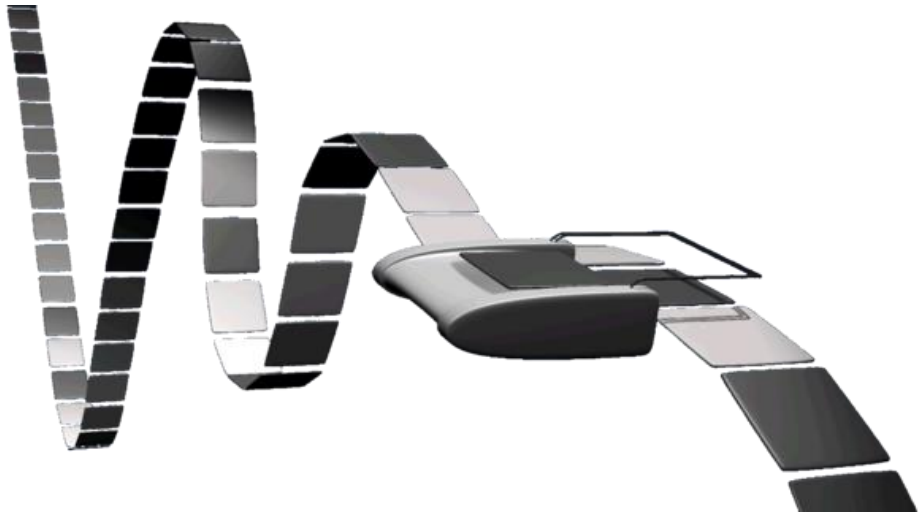
Remember that, all our assumptions are based on the fact that we deal with a UNIVERSAL TURING MACHINE (UTM).

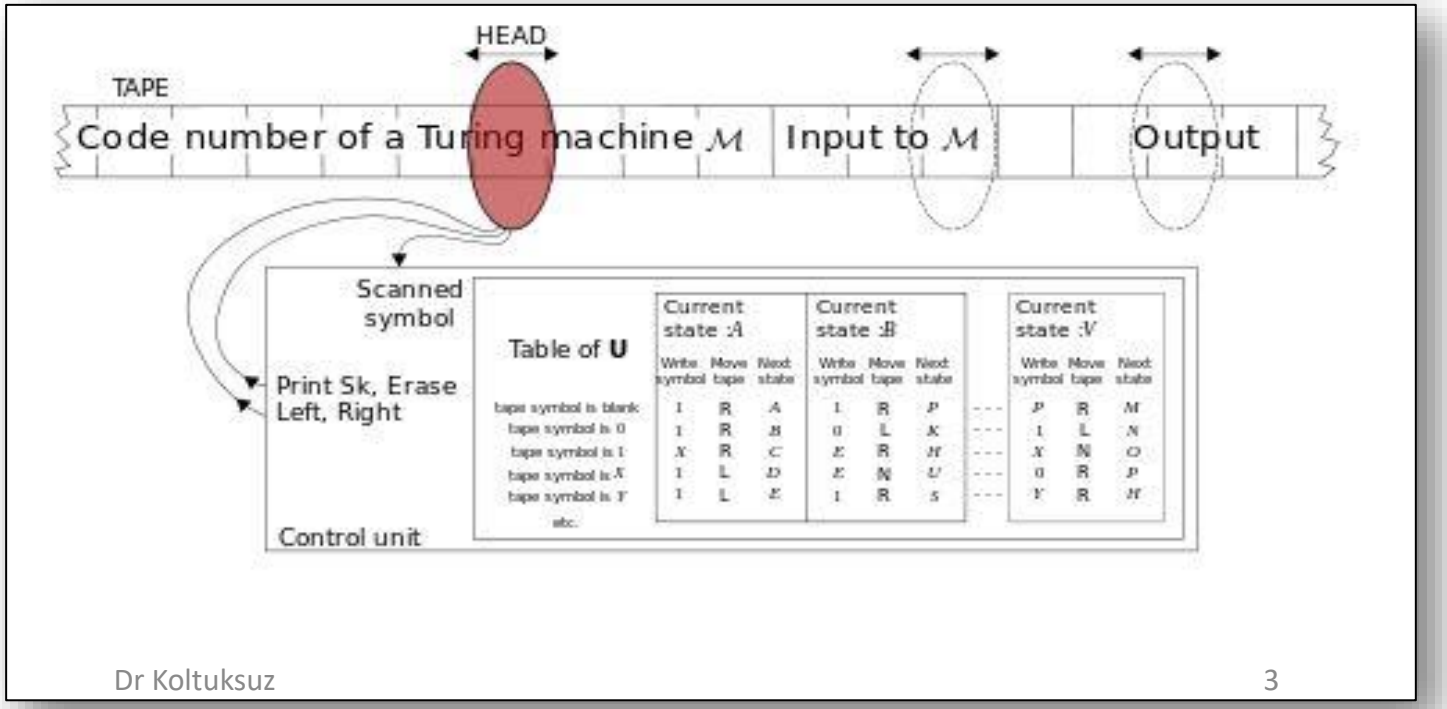
This is one model of computing that we know of.



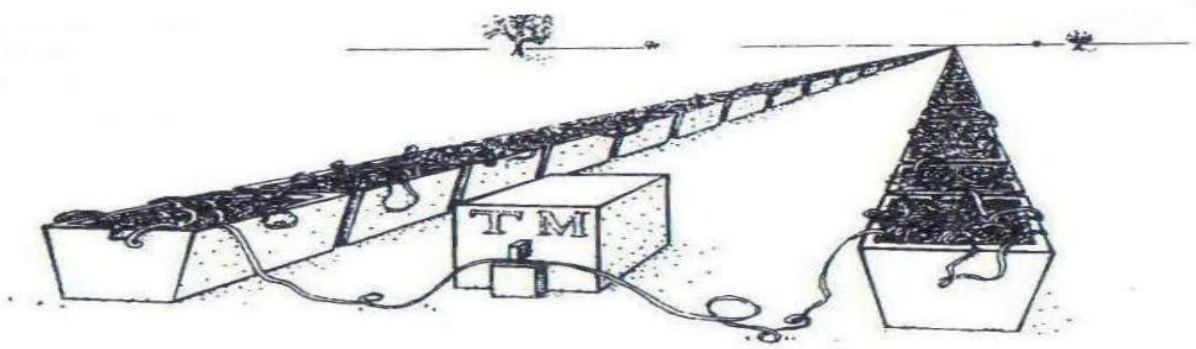
Universal Turing Machine (UTM), 1935

1. The memory consists of a tape divided into cells. Each square may be blank or "0" or "1" taken from a finite alphabet.
2. The head is able to process only one cell of a tape at a time.
3. The tape is the machine's general-purpose storage medium, serving as the vehicle for input and output, and as a working memory for storing the results of intermediate steps of the computation.
4. The tape may also contain a program of instructions = codes.
5. The tape itself is of unbounded length, unlimited working memory and unlimited time.





Universal Turing Machine (UTM), 1935



Formally; **TM=< Q, Σ, Γ, δ, q_0, □, F>** where

- Q is a finite set of internal states,
- Σ is an input alphabet that is a finite set of characters,
- Γ is a tape alphabet which is a finite set of symbols,
- δ is transition function,
- q_0 is the initial state, $q_0 \in Q$
- □ is blank symbol, □ ϵ Γ
- F is the final state, F ϵ Q

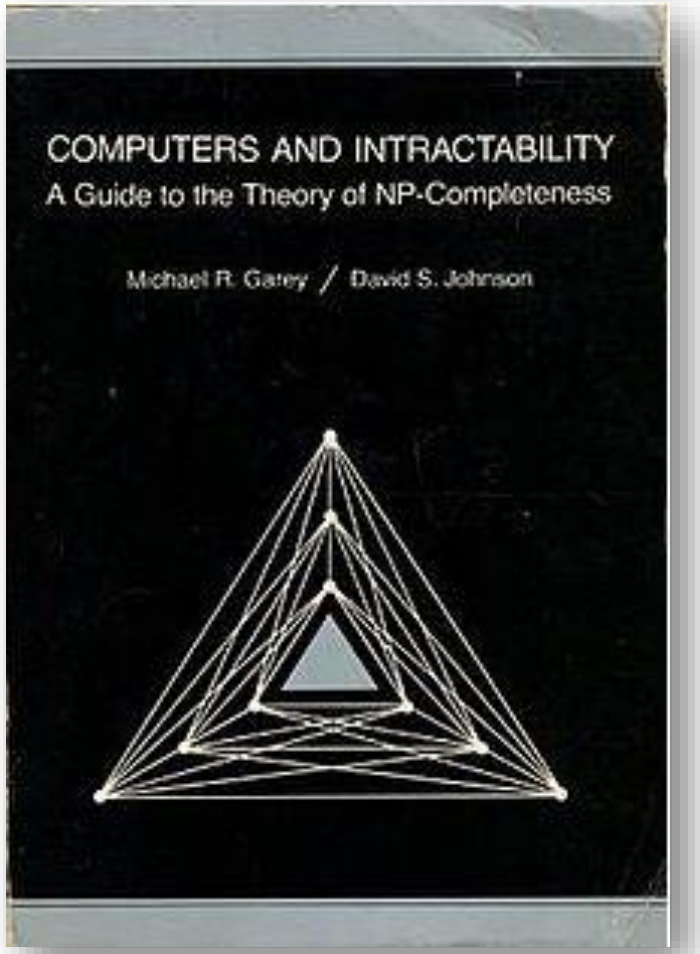
$$\delta: Q \times \Gamma \rightarrow Q \times \Gamma \times \{ L, R \}$$

- Current state of the control unit
- Tape symbol being read
- Next (new) state to be in
- Next (new) symbol to be written
- Direction of the head movement, Left or Right

How did it all begin with?

Michael Garey, David S. Johnson, Computers and Intractability: A Guide to the Theory of NP-Completeness, W. H. Freeman & Company, 1979, USA

It was the first book exclusively on the theory of NP-completeness and computational intractability



The List of unsolved problems in computer science

A) Computational Complexity

1. P versus NP problem
2. What is the relationship between BQP and NP?
3. NC = P problem
4. NP = co-NP problem
5. P = BPP problem
6. P = PSPACE problem
7. L = NL problem
8. PH = PSPACE problem
9. L = P problem
10. L = RL problem
11. Unique games conjecture
12. Is the exponential time hypothesis true?
    Is the strong exponential time hypothesis (SETH) true?
13. Do one-way functions exist?
    Is public-key cryptography possible?
14. Log-rank conjecture

Source: Wikipedia

The List of unsolved problems in computer science

B) Computational complexity

15. P versus NP problem
16. What is the relationship between BQP and NP?
17. NC = P problem
18. NP = co-NP problem
19. P = BPP problem
20. P = PSPACE problem

C) Polynomial versus non-polynomial time for specific algorithmic problems

21. Can integer factorization be done in polynomial time on a classical (non-quantum) computer?
22. Can the discrete logarithm be computed in polynomial time?
23. Can the shortest vector of a lattice be computed in polynomial time on a classical or quantum computer?
24. Can clustered planar drawings be found in polynomial time?
25. Can the graph isomorphism problem be solved in polynomial time?
26. Can leaf powers and *k*-leaf powers be recognized in polynomial time?
27. Can parity games be solved in polynomial time?
28. Can the rotation distance between two binary trees be computed in polynomial time?
29. Can graphs of bounded clique-width be recognized in polynomial time?
30. Can one find a simple closed quasigeodesic on a convex polyhedron in polynomial time?
31. Can a simultaneous embedding with fixed edges for two given graphs be found in polynomial time?

Source: Wikipedia

The List of unsolved problems in computer science

D) The Other algorithmic problems

32. The dynamic optimality conjecture: do splay trees have a bounded competitive ratio?
33. Is there a *k*-competitive online algorithm for the *k*-server problem?
34. Can a depth-first search tree be constructed in NC?
35. Can the fast Fourier transform be computed in $o(n \log n)$ time?
36. What is the fastest algorithm for multiplication of two *n*-digit numbers?
37. What is the lowest possible average-case time complexity of Shellsort with a deterministic, fixed gap sequence?
38. Can 3SUM be solved in strongly sub-quadratic time, that is, in time $O(n^{2-\epsilon})$ for some $\epsilon>0$?
39. Can the edit distance between two strings of length *n* be computed in strongly sub-quadratic time? (This is only possible if the strong exponential time hypothesis is false.)
40. Can X + Y sorting be done in $o(n^2 \log n)$ time?
41. What is the fastest algorithm for matrix multiplication?
42. Can all-pairs shortest paths be computed in strongly sub-cubic time, that is, in time $O(V^{3-\epsilon})$ for some $\epsilon>0$?
43. Can the Schwartz–Zippel lemma for polynomial identity testing be derandomized?
44. Does linear programming admit a strongly polynomial-time algorithm? (This is problem #9 in Smale's list of problems.)
45. How many queries are required for envy-free cake-cutting?
46. What is the algorithm for the lookup table that consistently generates playable mazes in the 1982 Atari 2600 game *Entombed* merely from the values of the five pixels adjacent to the next ones to be generated?
47. What is the algorithmic complexity of the minimum spanning tree problem? Equivalently, what is the decision tree complexity of the MST problem? The optimal algorithm to compute MSTs is known, but it relies on decision trees, so its complexity is unknown.

Source: Wikipedia

The List of unsolved problems in computer science

E) Natural language processing algorithms

48. Is there any perfect syllabification algorithm in the English language?
49. Is there any perfect stemming algorithm in the English language?
50. Is there any perfect POS tagging algorithm in the English language?
51. How can computers discern pronoun ambiguity in the English Language? (Also known as the Winograd Schema Challenge).

F) Programming language theory

52. POPLmark
53. Barendregt–Geuvers–Klop conjecture

G) Other problems

54. Aanderaa–Karp–Rosenberg conjecture
55. Generalized star height problem
56. Separating words problem

Source: Wikipedia

B. COMPUTER ENGINEERING

- ACTUALLY, THERE ARE MANY OTHER UNSOLVED PROBLEMS FROM COMPUTER ENGINEERING DOMAIN AS WELL.
- LET’S SEE AT LEAST SOME OF THEM

- Three walls that are blocking the way ahead:

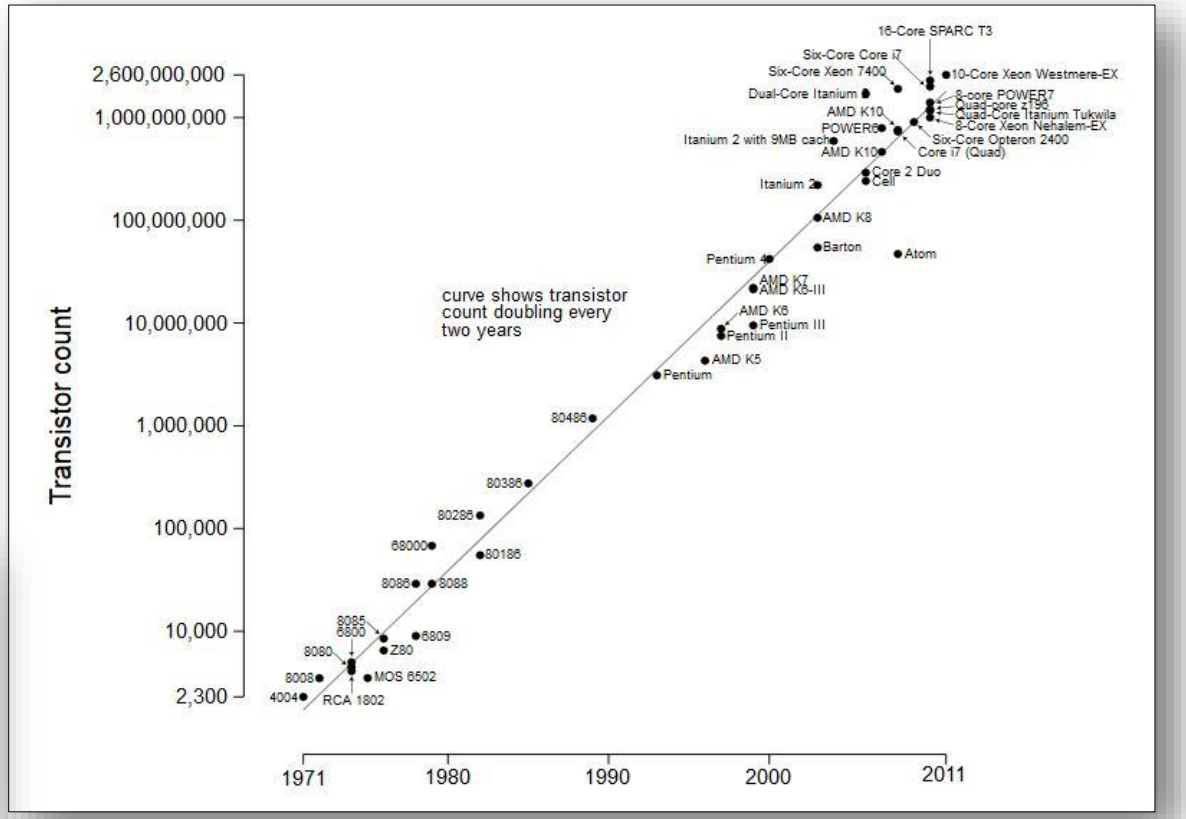
  #1 Wall of Processor Thickness

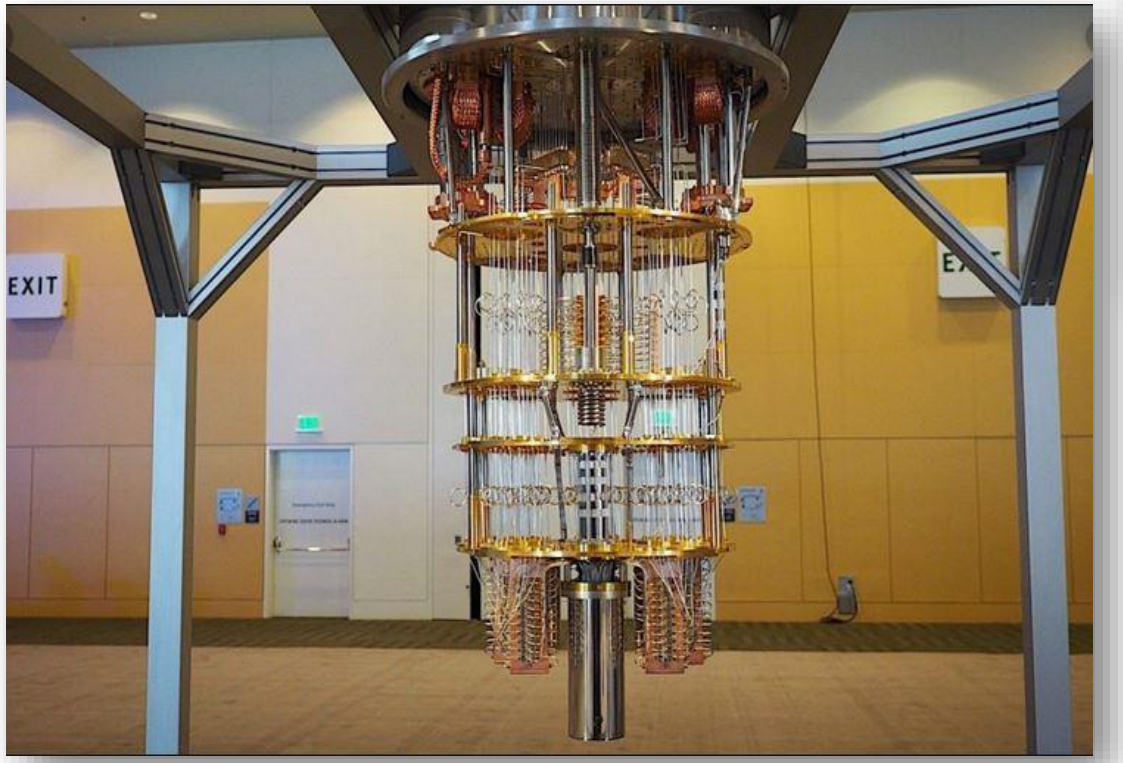
  #2 Wall of Clock Speed-Frequency

  #3 Wall of Memory – Memory Hierarchy

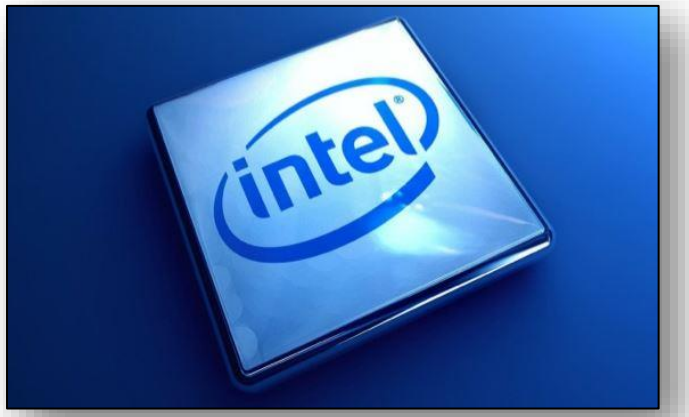
#1 Wall of Processor Thickness

- Moore's Law: every two years the number of transistors on a computer chip will double.
- In figures:
  - intel 8080, 1974, two inches long: 4500 transistors
  - Currently, 2015 at intel: 14nm thick silicon wafer: 4.5 billion transistors
- Moore's Law is dead, actually it has been dead for almost a decade now.
- Why or How? **Thickness wall: 4nm**, after that the silicon goes quantum. No more reliability of computing.



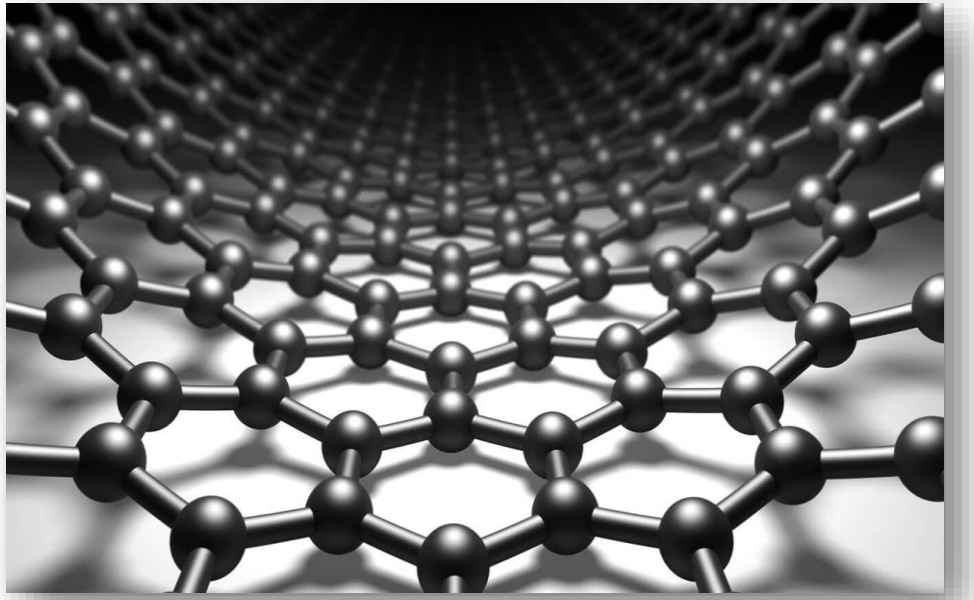


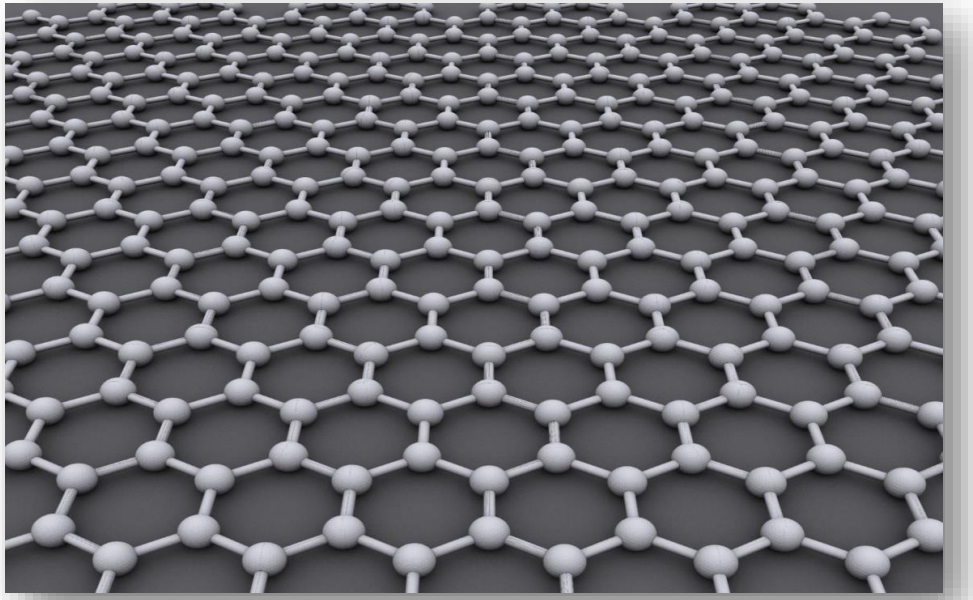
IBM Q
System One, 53 Qubit



#1 Wall of Processor Thickness

- **Solution?** IBM investigates carbon nanotubes and graphene as a computational substrate.
- Thus new materials needed.
- Enter Graphene. Graphene is 1 atom thick carbon sheets.
- No thickness wall at the moment. But it cannot reliably encode digital logic due to conductivity problem.

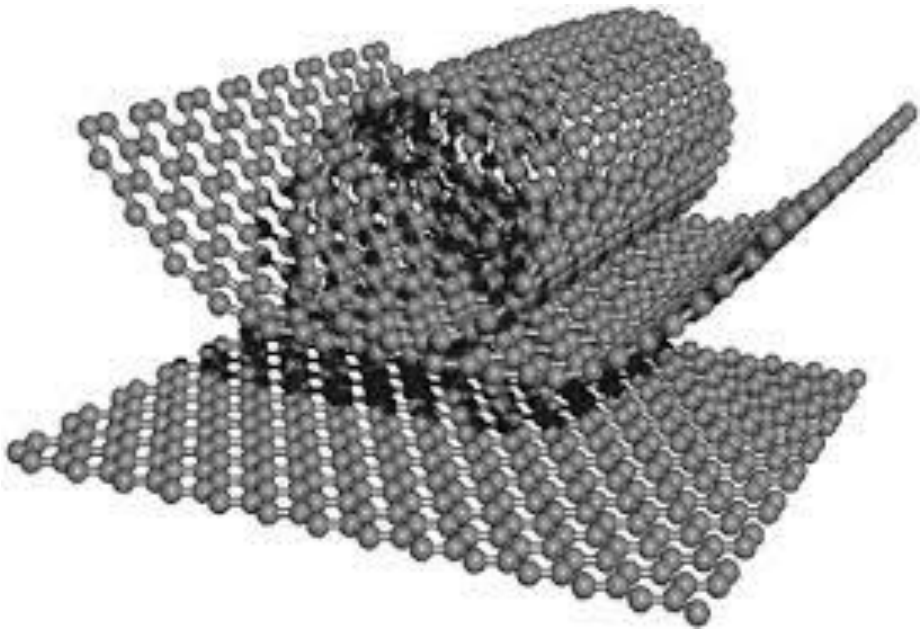


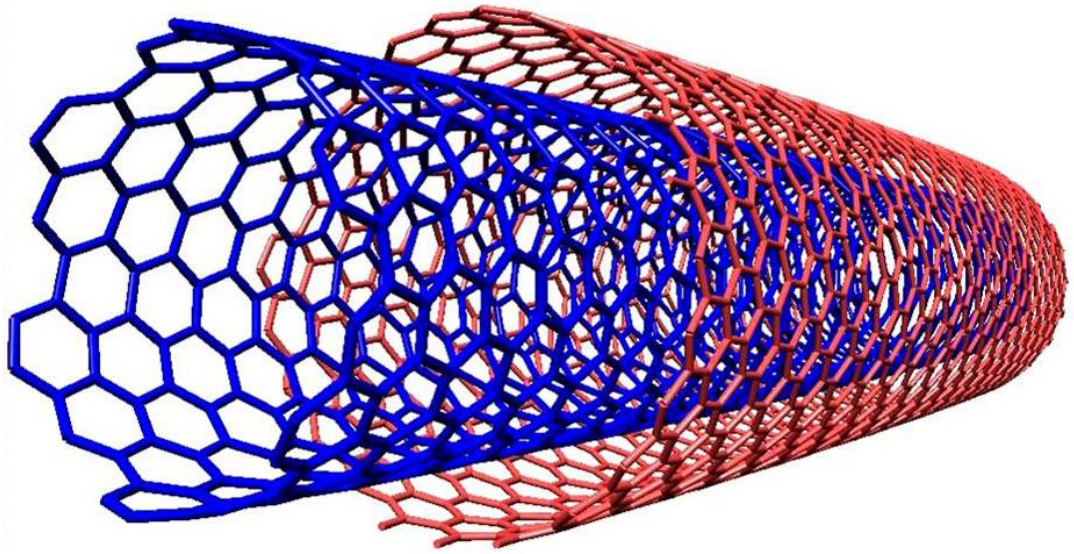


#1 Wall of Processor Thickness

- **Solution?** IBM investigates carbon nanotubes and graphene as a computational substrate.

- Carbon nanotubes, rolled graphene, it has semiconductor abilities like that of a silicone but has a tendency of losing it very easily. Many engineering problems there as yet.





#2 Wall of Clock Speed-Frequency

- **Frequency wall: 4Ghz** after that, the processor burns itself up from excessive heat. Hence; instead of 1 10Ghz processor, so how about 4, 8 or even 16, 3GHz processors working together? That is why we have multicore architectures.

- **Solution?** What about electrons—are those necessary? IBM and HP are also placing bets on photonics, which uses pulses of light instead of voltage.

#3 Wall of Memory – Memory Hierarchy

- **S(tatic)RAM-D(ynamic)RAM-HD-SSD** speed differences,
- HP has built a prototype computer, code-named “the Machine,” that leverages the power of a long-sought missing link of electronics: the memristor.
- This component—mathematically predicted decades ago but only recently developed—allows the storage and random-access memory (RAM) functions of computers to be combined.

#3 Wall of Memory – Memory Hierarchy

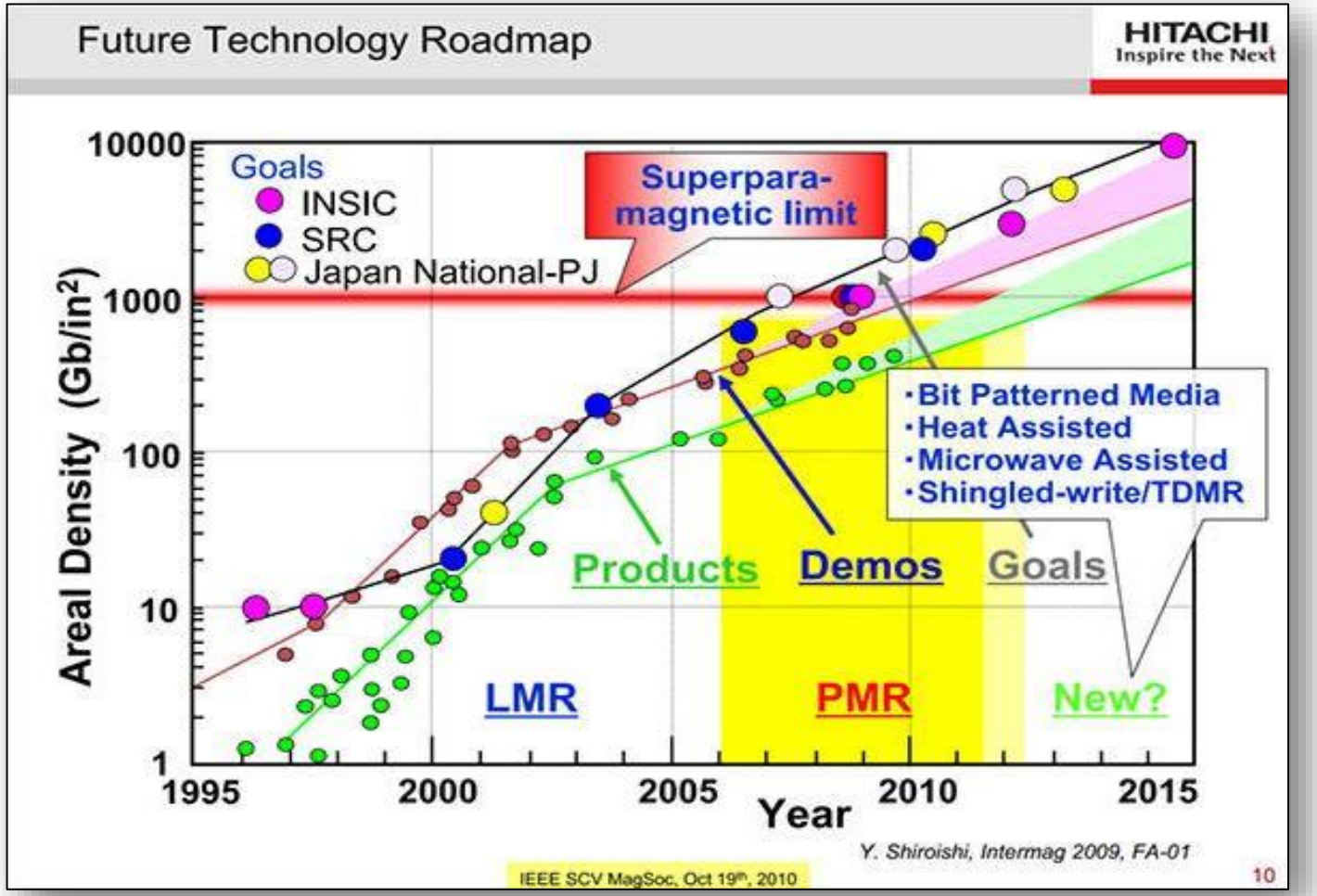
- The CPU as a computer's "brain" would become more accurate with memristors instead of transistors because the former actually work more like neurons: they transmit and encode information as well as store it.
- Combining volatile memory and nonvolatile storage in this way could dramatically increase efficiency and diminish the so-called von Neumann bottleneck, which has constrained computing for half a century.

#3 Wall of Memory – Memory Hierarchy

- Winchester Hard Disk Technology:

Parameter	1956	2017
Formatted capacity	3.75 megabyte	12 terabyte
Physical volume	1.9 m^3	34 cm^3
Weight	910 kg	62 g
Ave. access time	Appx 600 milliseconds	2.5 - 10 ms; RW depending on RAM type
Price	US$9,200 per megabyte	US$0.032 per gigabyte
Data recording density	2,000 bit per inch square	1.3 terabit per inch square
Ave. lifetime	~2000 hours	~22500 hours (~2.5684 years)

Can you guess what this is?
It's a hard disk drive (HDD) with 5 MB storage back in 1956. In 1956 IBM launched the 305 RAMAC, the first computer with a HDD. The HDD weighed a ton and stored 5 MB of data. Start appreciating your 8 GB memory stick!

#3 Wall of Memory – Memory Hierarchy

- HD Technology: Seagate introduced a 6TB disk in early 2014 and, announced an 8TB disk in 2015 based on Shingled Magnetic Recording (SMR).
- Now, it is 12TBs and even more…
- **But still not enough.**